



ARTIFICIAL INTELLIGENCE

State space search

ABSTRACT

State space search is the process of searching for a sequence of actions that take from an initial state to a goal state to formulate a solution for a given problem.

3.1 State space

In the theory of discrete dynamical systems, a state space is the set of values which a process can take. For example, a system in queuing theory recording the number of customers in a line would have state space $\{0, 1, 2, 3, \dots\}$.

In games, the state space is the set of all possible configurations within the game which can be reached by legal actions. For instance, in chess, it consists of all the possible board configurations. Within this state space there is the subset of configurations which are valid according to the rules of chess.

A game's total state space is often readily calculated whereas finding the subset of valid configurations may be a considerable challenge. The size of a game's state space is related to its complexity. For some systems, the state space can be so huge that even for the fastest computer on earth, it make take thousands of years to explore all the states.

Here are the sizes of state space for some problems:

Problem	State space size
Theorem proving	Infinite
Chess	10^{120}
Checkers	10^{40}
8-puzzle	181,440

3.2 Problem solving by searching: State space search

- A wide range of AI problems can be formulated as searches.
- State space search is the process of searching for a sequence of actions that take from an initial state to a goal state to formulate a solution for a given problem.

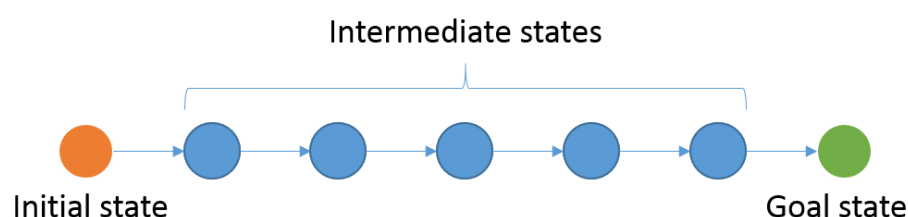


Figure: State space search

Search can be applied to many problems:

- Solution to a puzzle.
- Shortest path on a map.
- Proof of a theorem
- Sequence of moves to win a game
- Sequence of moves to achieve a desired state

3.3 Formulating state space search

State space search can be applied for those systems which have a set of distinct states and the system can go from one state to another by applying an action from a set of legal actions/operators. The search is started from a given initial state and run until a desired goal state is reached.

In summary, A problem is a tuple (S, s, A, ρ, G, P) where

- S is a set of states
- $s \in S$ is the initial state
- A is a set of actions
- $\rho: S \times A \rightarrow S$, is a partial function, that tells you for each state, which actions will take you to which states.
- $G: S \rightarrow \text{bool}$, is the goal test function, which tells you whether a state is a goal state or not.
- $P: S \times (A \times S)^* \rightarrow \text{real}$, is the path cost function. A path is a sequence $\{s_0 a_1 s_1 a_2 \dots a_k s_k\}$ such that $\forall i \in \{1..k\}, \rho(s_{i-1}, a_i) = s_i$. Usually, but not always, this is the sum of the "step costs" assigned by a simpler step cost function (of type $S \times A \times S \rightarrow \text{real}$).

The task is to find:

- a sequence of actions that leads from the initial state to a goal state

As a frank explanation, a state space search is represented by:

- A set of states: the state space.
- An initial state.
- A set of operators/actions
- A set of goal states, or a single goal state
- An optional path cost function

3.4 The search agent

An agent that tries to come up with a sequence of actions that will bring the environment into a desired state is called the search agent. Generally the search agent is a goal-based agent.

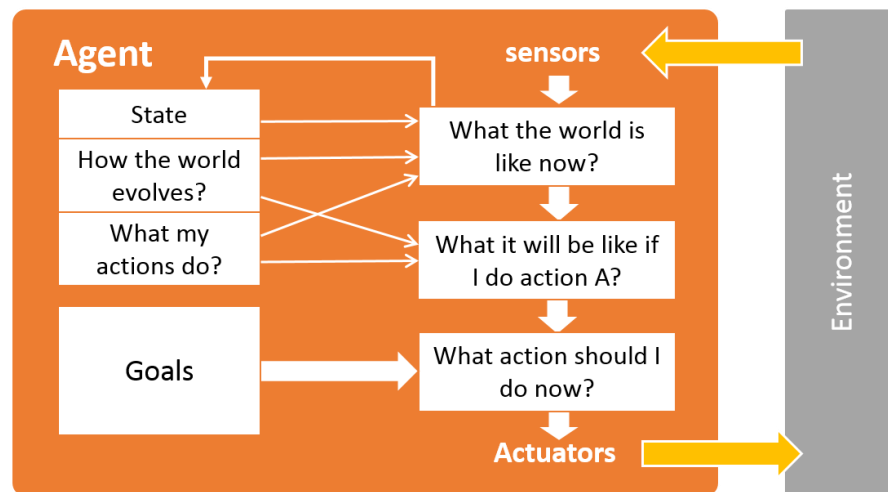


Figure: a goal-based agent

3.5 Some Classical search problems

3.5.1 The water-jug problem

Suppose you are given two jugs with no measuring marks, a 3-gallon one and a 4-gallon one. There is a pump to fill the jugs with water. How can you get exactly 2 gallons of water into the 4-gallon jug?



Figure: The water-jug problem

Formulating water-jug problem as a state space search problem:

State representation: We can represent the states as (m, n) where m and n represents the current amount of water in jug 1 and 2 respectively. $m \in \{0, 1, 2, 3\}$ and $n \in \{0, 1, 2, 3, 4\}$.

Initial state: The initial state is $(0, 0)$.

Goal state: The goal state is $(x, 2)$, for $x \in \{0, 1, 2, 3\}$

Actions: There are three actions we can apply to any of the jugs: fill, empty and pour from one to another.

Path cost: No path cost function for this problem.

3.5.2 The 8 puzzle problem

The 8-puzzle is a square board with 9 positions, filled by 8 numbered tiles and one gap. At any point, a tile adjacent to the gap can be moved into the gap, creating a new gap position. In other words the gap can be swapped with an adjacent (horizontally and vertically) tile. The objective in the game is to begin with an arbitrary configuration of tiles, and move them so as to get the numbered tiles arranged in ascending order either running around the perimeter of the board or ordered from left to right, with 1 in the top left-hand position. An example scenario is shown below.

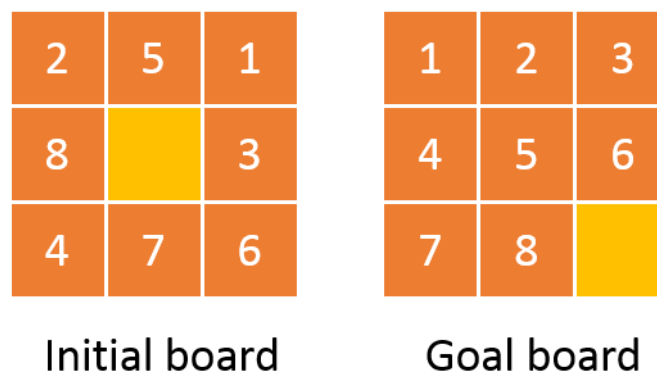


Figure: 8 puzzle problem

3.5.3 N Queen or 8 Queen problem

Place N queens in a $N \times N$ chessboard so that no queen can attack another.

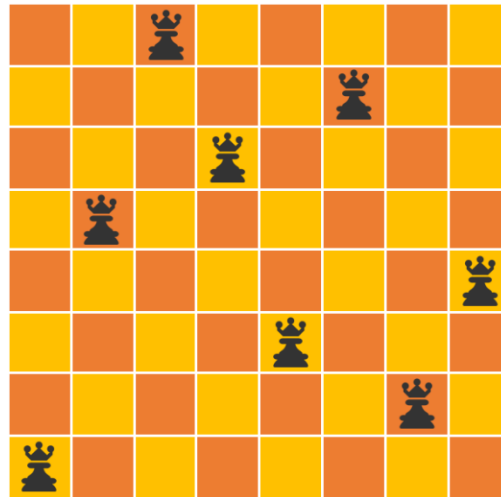


Figure: A 8 queen solution

3.5.4 Route finding problem

Find the quickest route from Arad to Bucharest.

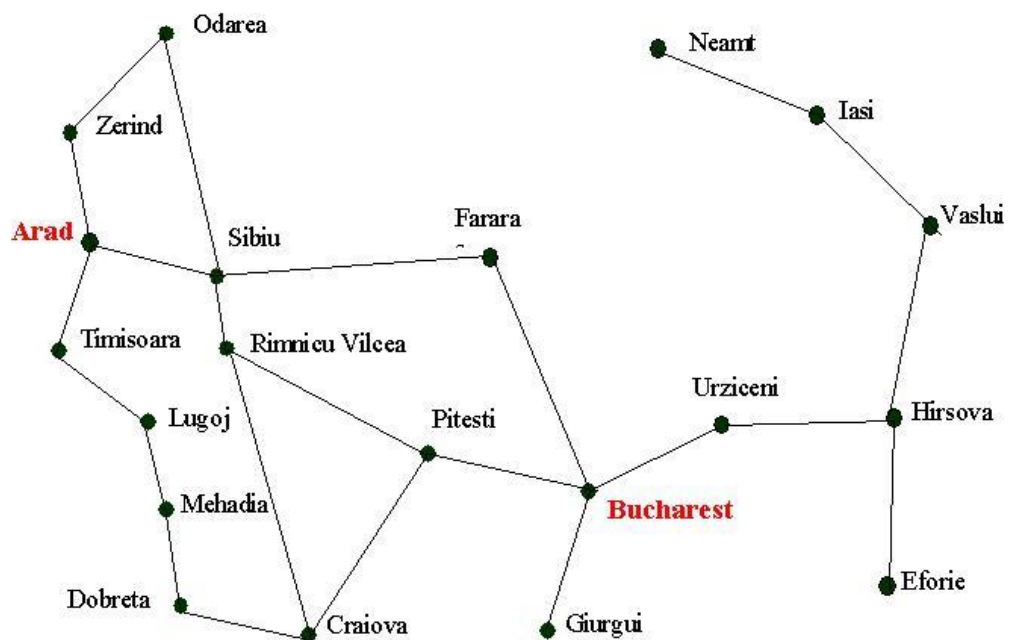


Figure: Route finding problem

3.5.5 More examples

- Puzzles: Rubik's Cube, Towers of Hanoi etc.
- Detecting bugs in software and hardware.
- Non-player-characters in computer games.
- Travelling Salesman Problem (TSP)

- Attack planning: Finding a hack into a secured network. Used for regular security testing
- Query optimization in databases.
- Sequence alignment in Bioinformatics.
- And many more...

3.6 Problem types

There are basically three types of search problems.

- **Representation problems:** Only need a representation of the system in a desired state, e.g. crossword puzzles.
- **Goal finding problems:** We only know the properties that a goal state should have, but we don't even know if any goal states exist. We just need to find a state that satisfies certain constraints for being a goal state. We don't care what action sequence gets us there. In these problems, optimality is concerned with the "best state". Examples: N-queens, integrated circuit layout and most other kinds of optimization problems.
- **Action sequence finding problems:** We know the state space in advance. We know which states are goals. We have to find the sequence of actions that get us to a goal state. In these problems, optimality is concerned with "cheapest path". Examples: mathematical theorems, Towers of Hanoi, 8 puzzle, water-jug problem and many other robotics problems.

3.7 State space representation

3.7.1 The state space graph

For a state space search problem:

- All the states of the system are represented as nodes of a graph.
- An action that can change the system from one state to another (e.g. a move in a game) is represented by an edge from one node to another.
- Edges may be unidirectional (e.g. Tic-Tac-Toe, chess, can't go back) or bi-directional (e.g. geographic move).

- It may be possible to reach the same state through many different paths.

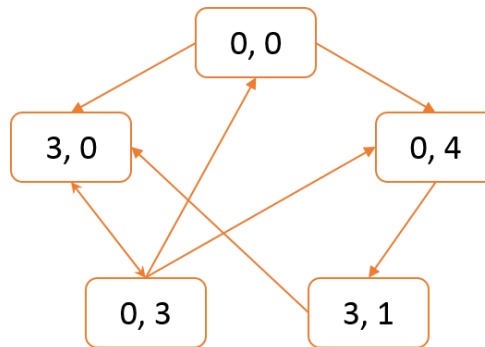


Figure: A partial view of the state space graph for the classical water-jug problem with jugs of sizes 3 and 4 gallon

3.7.2 The search tree

At any state, the search agent has to choose from a set of legal actions which will take to a state which is likely on the path of the goal state. As often graphs contain cycles, the search agent can come back to a state it previously explored, and thus traversing the same path over and over again. For efficient searching, it is desirable that the search agent explores a node only once unless there is some need of backtracking. So, in general, the explored nodes in a state space graph constructs a spanning tree rather than a sub-graph.

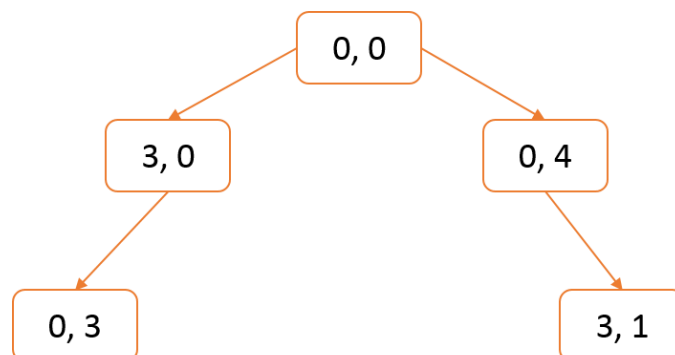
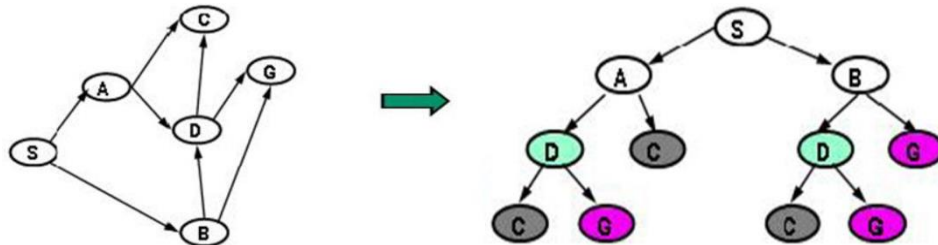


Figure: A partial view of the state space search tree the search agent actually explores for the classical water-jug problem with jugs of sizes 3 and 4 gallon

We can turn graph search problems (from S to G) into tree search problems by:

- replacing undirected links by 2 directed links
- avoiding loops in path (or keeping track of visited nodes globally)



In the search tree:

- root node represents the initial state
- children of a node represents successor states
- fringe of the tree represents the states not yet expanded

3.8 Search strategy

3.8.1 Construction of the search tree

- The initial state is the root of the tree.
- For each un-expanded node in the tree, successors are generated. This is called **node expanding**. For the successors' point of view it is called **node exploration**, i.e. the node is expanded and the successors are explored.
- The successors of a node is attached to the tree as the children of the node.
- We expand one node at a time. We have to decide which node to expand next from all the un-expanded nodes in the tree. How we make this decision is the essence of the **search strategy**.

3.8.2 Algorithm evaluation

We evaluate the searching algorithms depending on four criteria:

- **Completeness**: Are they guaranteed to find a solution?
- **Time complexity**: How long do they take in search time?
- **Space complexity**: How much storage is required?

- **Optimality:** When there are several solutions, does it find the best one?

Time and space complexity are measured in terms of:

- ***b***: maximum branching factor of the search tree
- ***d***: depth of the least-cost solution
- ***m***: maximum depth of the state space (maybe ∞)

3.9 Recommended reading

<http://cs.lmu.edu/~ray/notes/problemsolving/>

State Space Search is an AI problem-solving technique that models a problem as a search through a graph or tree. The goal is to find a path—a sequence of actions—that moves the problem from a starting configuration to a desired final configuration (the goal).

How It Works

1. **State:** Represents a snapshot of the problem (e.g., a specific arrangement of puzzle).
2. **State Space:** The entire collection of a possible states.
3. **Action/Operator:** A legal move that transforms one state into a successor state (e.g., moving a tile).
4. **Search Algorithm (Strategy):** Starting from the Initial State, an algorithm (like BFS, DFS, or A*) systematically explores the state space, generating successor states until a path leading to the Goal State is found.